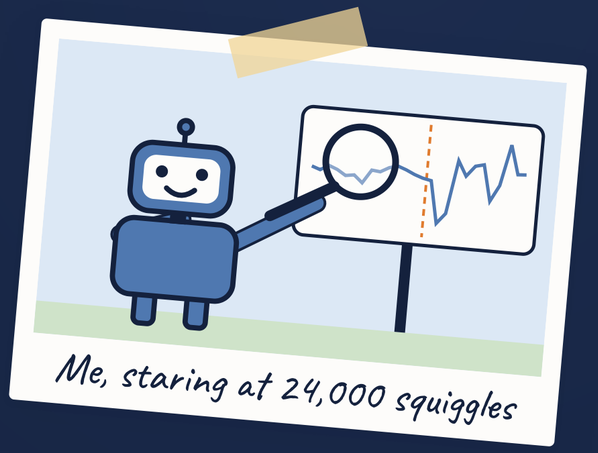
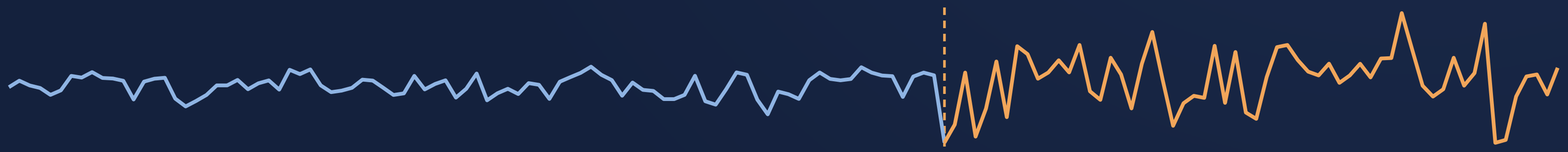
KU SBB · TEAM BLITZICT

Spotting when a pattern changes



Four attempts at the same problem: what each one did, why it scored better, and how I had to nerf my own best model to follow the rules



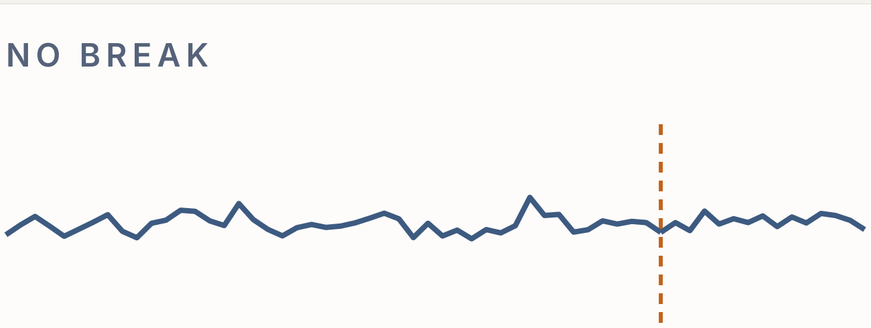
BACKGROUND

The task

Each example is a **time series**: a list of numbers recorded one after another, like a daily temperature or a machine's sensor reading.

Each one has a **marked point**. The question: **did the behaviour change at that point?** That change is called a **structural break**.

I get 24,000 practice series with the answer, and must give a probability for 4,000 new ones.



SCORING

How I am scored: AUC

Pick one series that has a break and one that doesn't. **AUC is how often my system gives the break series the higher probability.**



0.50
A coin flip

0.88
My best: right 88 times out of 100

1.00
Perfect every time

Two scores appear in this deck: **public** is Kaggle's score on the test series; **local** is my own check on the practice series.

OVERVIEW

The four tries at a glance

		Public score (axis starts at 0.82)
try1	Compare the numbers before and after the point	0.833
try2	Also ask how the numbers were produced	0.875
try3	The full model: a large committee, then corrections	0.881 · best
try4	try3 cut down to fit the time limit	0.876 · final

try3 is my best model, but it is too slow for the time limit. So I nerfed it myself: try4 is the nerfed version, and my final model.

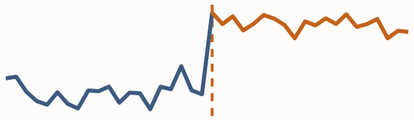
TRY1 · THE IDEA



Measure both halves and compare

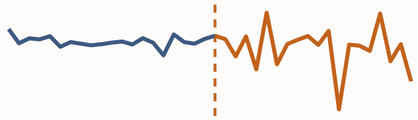
Cut each series at the marked point. Describe the “before” half and the “after” half with many measurements, then look at the differences.





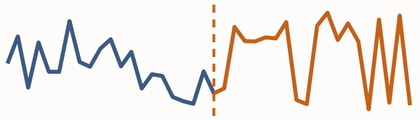
Level

Is the average higher or lower after the point?



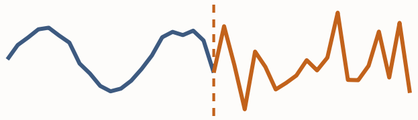
Spread

Do the values swing more widely?



Shape

Do the two halves look like they come from the same mix of values?



Memory

Does today’s value still predict tomorrow’s as well as before?

TRY1 · HOW IT DECIDES · 0.833

977 measurements, two models that vote

01

Features

977 numbers per series, each one a before-vs-after comparison like those on the last slide.

02

Model

A program that learns from the 24,000 answered examples. Mine are “decision trees”: thousands of learned yes/no questions, such as “did the spread double?”

03

Vote

Two tree programs (XGBoost and LightGBM) each rank the series; I average their rankings.

0.833 Most of my time went into try1. It also got the lowest score of the four. A solid start, I tell myself.

TRY2 · THE IDEA



Ask how the numbers were produced

try1 asked “do the halves look different?”. try2 also asks “**did the process making the numbers change?**” I fit simple descriptions of that process to each half and compare them.

01 Rhythm
a repeating cycle that changes speed or stops

02 Trend
a steady climb that bends or reverses

03 Jumps
sudden spikes appearing or vanishing

04 Calm vs stormy
quiet and noisy periods switching

05 Noise
the random part getting louder or changing type

06 Momentum
how strongly one value pulls the next

TRY2 · RESULT · 0.875

More and better measurements, one model

3,625 features per series: try1's comparisons, the same comparisons on windows of different sizes around the point, and the new process measurements.

Fed to a single tree model (LightGBM).

VS TRY1

+0.042

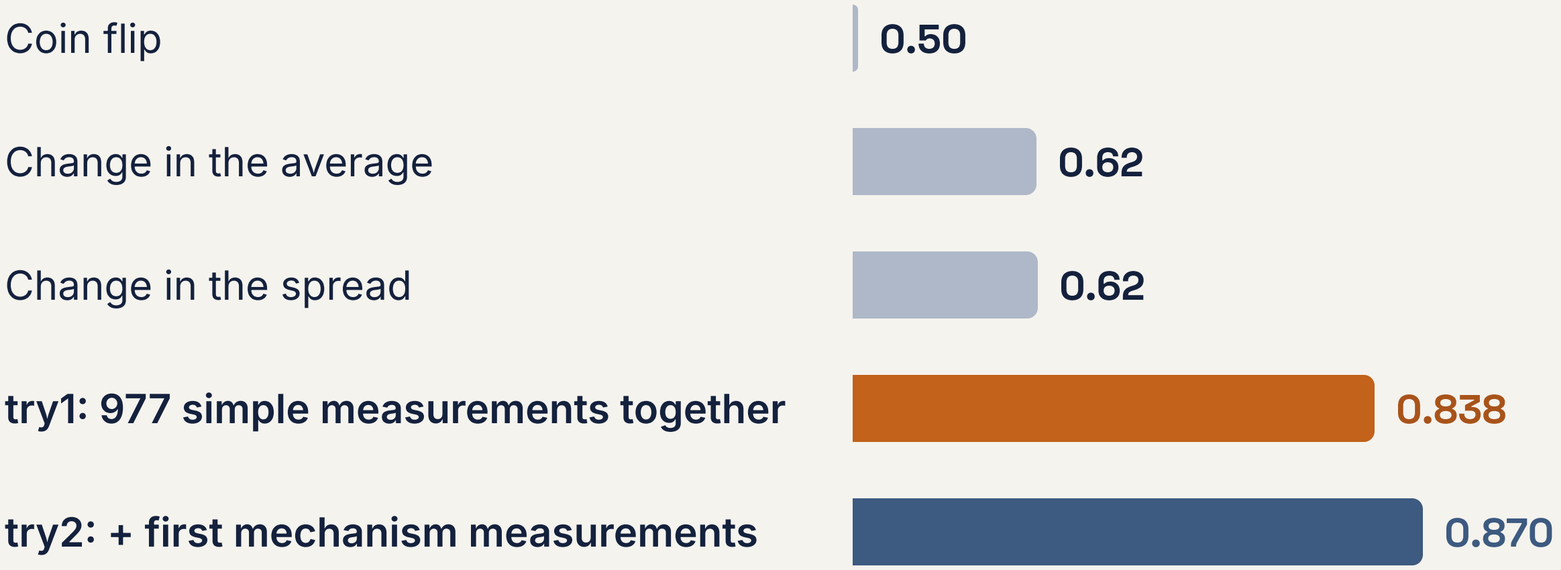
over try1, the biggest single jump of the project

Lesson: describing how the data was made beats generic statistics (and beats a week of polishing try1).

TRY1 → TRY2 · WHAT THE NUMBERS SAY

Simple comparisons barely beat a coin flip

Held-out AUC on the 24,000 practice series (axis starts at 0.50)



INVISIBLE BY EYE

39%

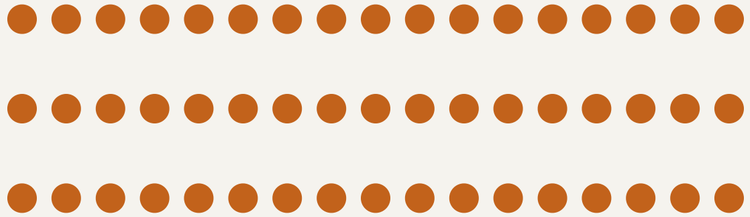
of the real breaks move the average by less than 0.1 standard deviations.

Eyeballing: 0.62. Describing how the numbers were made: 0.87.

TRY3 · THE FULL MODEL · 0.881

A committee of 51 experts

One model has blind spots. try3 trains **51 different models**, each with a different way of looking, and combines their opinions, like asking a panel instead of one person.



51 MODELS

About 5,100 measurements in total. Spoiler: one of these 51 ends up doing most of the work.

Tree models

Yes/no-question models, each given a different set of measurements

Neural networks

Models loosely inspired by the brain, good at blending many weak signals

Sequence readers

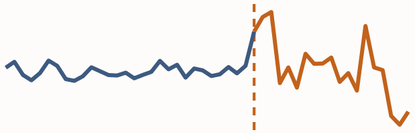
Networks that read the raw numbers directly, with no hand-made measurements

TRY3 · INGREDIENT 1



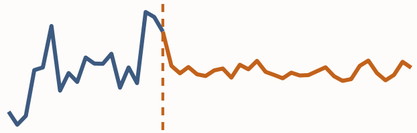
Look at each series 8 ways

A break is still a break if you play the series backwards or upside down. So I make extra copies of every series and let models learn from all of them.



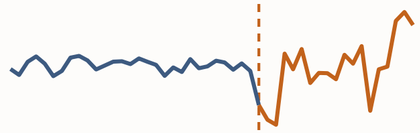
Original

As recorded



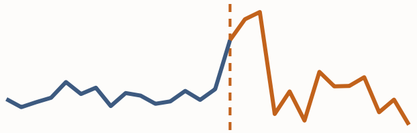
Reversed

Played backwards



Flipped

Upside down, and both



4 crops

Zoomed in around the point

When scoring, these models answer for every copy and I average, like asking someone to judge a photo and its mirror image.

TRY3 · INGREDIENT 2

The specialist that carried the team

WHAT IT IS	One XGBoost model, called MECH3C12V in my code. One of try3's 51.
WHAT IT SEES	Only "how was it produced" measurements: 11 mechanism tables (rhythm, trend, momentum, noise tests…), 2 spike laws and the 36 noise-prior columns.
EXTRA PRACTICE	Every training series also appears as 2 zoomed-in crops around the point, each with its own measurements.
FAIR TESTING	Trained 5 times on 4/5 of the data. Its noise priors are learned from those 4/5 only.

ALONE, HELD-OUT AUC

0.8748

All 51 models + corrections: 0.8791. This one model gets 99.5% of the way.



TRY3 · INGREDIENT 3

Combine, then fine-tune the answer



1. **Blend**: combine the 51 opinions into one, giving more weight to the experts that proved more reliable.

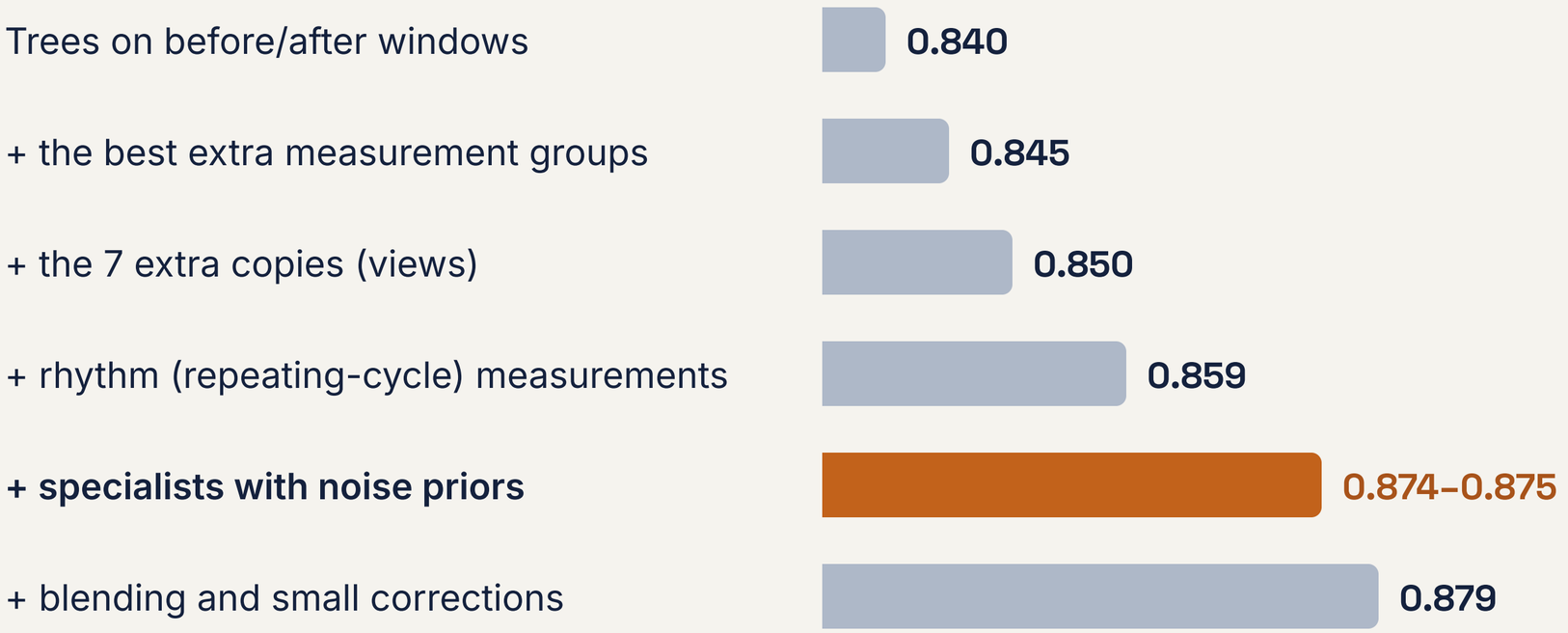
2. **Small corrections**: dozens of small adjustment steps, each nudging the blended answer using one extra signal. A step is kept only if it improves the score on data it didn’t learn from.

3. **Practice on made-up data**: one more neural network, trained on series I generated myself (so the answers are known), adds a final correction. This took me from 0.879 to 0.881.

TRY3 · WHAT EACH PART ADDED

What each ingredient was worth

Local score as each part was added (axis starts at 0.835)



BIGGEST GAIN

+0.015

from the specialists: the biggest jump on the ladder. One idea beat weeks of tinkering.

TRY3 · THE CATCH

The best score is expensive to run

473

separate steps to train it, many of which must wait for the one before

~1 day

of computer time to retrain it from scratch, even with two graphics cards

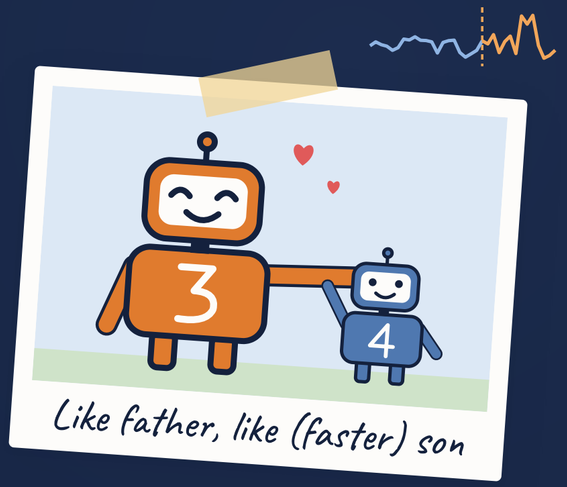
hours

to score new series. The rules allow 30 minutes on 2 processor cores with 8 GB of memory

So my best model could not be my final submission. Time to nerf my own model: how much of it do I actually need?

TRY4 · 0.876 · MY FINAL MODEL

try3, nerfed to fit the rules



THE RULE, FROM SCRATCH:

30 min
to score 4,000 new series

2 cores
of processor

8 GB
of memory

The goal: nerf it as gently as possible, so it still fits the rule. Three kinds of cuts:

- **Cut 1 · fewer models**
- **Cut 2 · fewer measurements**
- **Cut 3 · less wasted computing**

CUT 1 · THE COMMITTEE

From 51 experts to 3

TRY3

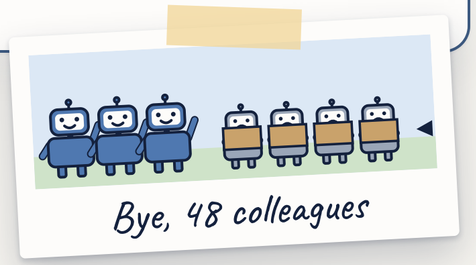


51 models of five kinds, including slow neural networks and sequence readers, then dozens of correction steps and a made-up-data network.

TRY4



Only fast tree models: 2 LightGBM and 1 XGBoost. Their rankings are averaged. No neural networks and no correction steps.



Tree models do most of the work and run in seconds. The other 48 cost hours for the last fraction of a point, so they were let go.

CUT 2 · THE MEASUREMENTS

From about 5,100 measurements to 136

100

I ranked every measurement by how much it helps and kept only the **top 100**. The rest add very little once the best ones are in.

+36

I kept the **noise priors** from try3's specialists, its most valuable ingredient.

−4

I dropped the **4 slowest groups** of measurements entirely: about 7 minutes saved for a loss of just 0.0004. A bargain.

CUT 3 · WASTED COMPUTING

Only calculate what gets used

01

Skip unused columns

Each measurement group computes dozens of numbers at once. try4's versions compute only the ones the models read, with identical results.

02

Fewer steps

77 measurement steps instead of try3's 473 training steps, because only the needed groups are built.

03

Work in parallel

Steps that don't depend on each other run at the same time on all available processor cores.

I (*diff cmd from google) checked that the trimmed calculations give exactly the same numbers as the full ones.

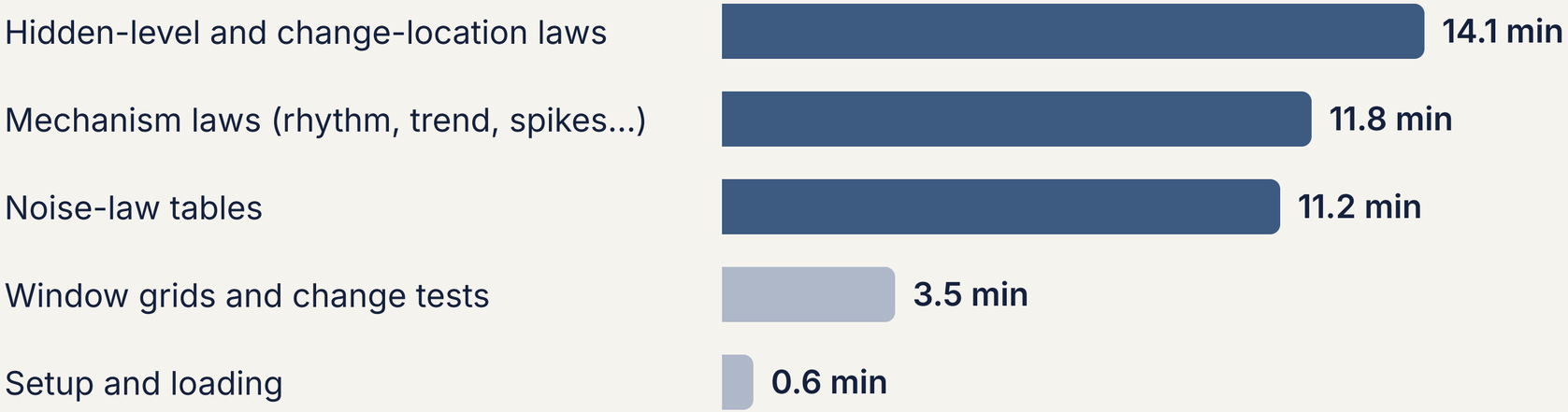
CUT 3 · THE RESULT

Where the 21.8 minutes go

Scoring 4,000 new series from raw data, Kaggle limited to 2 cores



Processor time by step family (2 cores work in parallel)



PEAK MEMORY

2.6 GB

of the 8 GB allowed

41 processor-minutes across 41 steps, 2 at a time, fit into 21.8 minutes. I had 8 minutes to spare. I did not spend them.

The exact columns try4 picked

3,589
candidate measurements

100
picked in each round

39
picked in all 5 rounds

+36
noise-prior columns

Round 1’s 100 picks, by family



+ 21 more families with 42 picks

HOW THEY WERE PICKED

In each round, a LightGBM trained on that round’s 4/5 ranks every candidate by how much it helps. The top 100 stay.

192 different ones across all 5 rounds.

INSIDE TRY4 · WHAT THE MODEL RELIES ON

The 8 measurements that matter most

	Share of the model's decisions	AUC alone
Noise volume test · fits better if each side gets its own noise volume?	17.9%	0.82
Restart test · one hidden process, or two split at the point?	10.2%	0.76
Change at the point · any change detected right at the point	5.7%	0.79
Noise prior · how often noise like this meant a break	3.1%	0.79
Change is local · stronger at the point than far before or after	3.0%	0.80
Rhythm phase · useless alone, sharpens the others	2.4%	0.50
Restart, new volume · same process, new volume after the point	2.0%	0.76
Momentum law · did the pull from one value to the next change?	1.9%	0.78

Together, the 36 noise-prior columns make 9.9% of the decisions.

One real break, explained

A HELD-OUT SERIES · ROUND 4

972 points, marked point at 440

Average moves **0.025 SD**

Spread changes **8%**

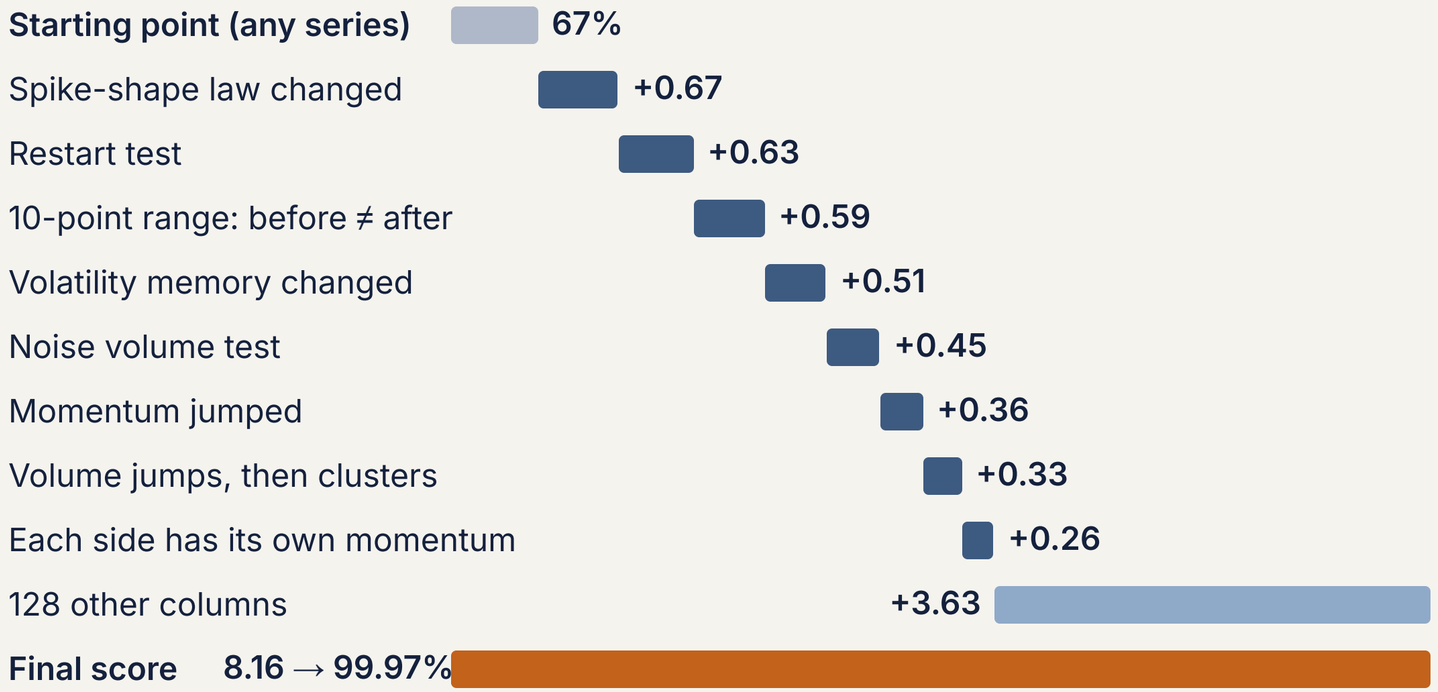
By eye: nothing happens.

99.97%

try4 says break: top 0.1% of all 24,000

Truth: a break ✓

How each measurement pushed the score (log-odds)



Each of the 8 values is higher than at least 99.9% of the no-break series in the same round.

HOW I TESTED IT · 5 ROUNDS

Whole series in, whole series out

I split the 24,000 series into **5 groups of 4,800 whole series**. Each round trains on 4 groups and tests on the 5th, so every series is tested exactly once, by a model that never saw any of its points.



Each box is 4,800 complete series, never pieces of one. Picking the top 100 measurements and fitting the noise priors also use only the 4 “train” groups.

HOW I TESTED IT

Held out whole, fitted on training only

WHOLE SERIES

The 24,000 series go into 5 groups of 4,800. A held-out series is held out **whole**: none of its points is ever trained on. That copies the real test, which is 4,000 brand-new series.

TRAINING ONLY

In each round, picking the top 100 columns, fitting the 36 noise priors and training all 3 models use the **4 training groups only**. Tree counts are fixed, so the held-out group isn't even used to decide when to stop.

NO LABELS INSIDE

Every measurement is computed from the series itself, without labels. 2 tables that were fitted on all the labels are left out.

A LEAK I CAUGHT

~~0.8774~~

Picking the columns once, with every round's labels, scored 0.8774. Too good to be true. Picked inside each round, the honest score is 0.873.

DID I FOOL MYSELF?

Held-out: **0.8723**. Kaggle, 4,000 unseen series: **0.8758**. No.

TEST RESULTS · 5 HELD-OUT ROUNDS

Steady in every round, and on Kaggle

AUC (axis starts at 0.85)



THREE MODELS, ONE VOTE

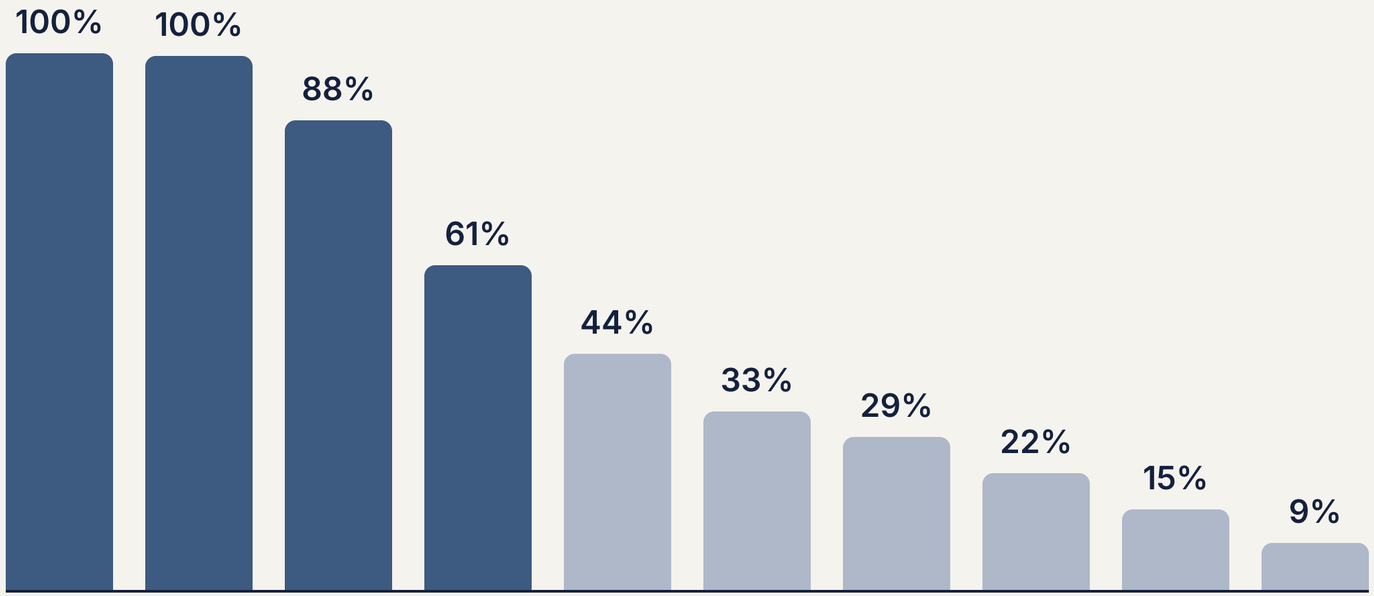
LightGBM	0.8718
LightGBM (seed 2)	0.8717
XGBoost	0.8714
Averaged ranks	0.8723

Kaggle scored it a bit higher than I did. I'll take it.

TEST RESULTS · HELD-OUT PREDICTIONS

When try4 is sure, it is right

Share of real breaks in each tenth, sorted by try4's score



The top half of the list holds 78% of all breaks.

4,790 / 4,800

of the series it is most sure about (top 20%) really were breaks.

8.9%

of the 2,400 it is least sure about were breaks. The coin-flip rate is 50%.

TEST RESULTS · ERROR ANALYSIS

Where try4 gets it wrong

SERIES LENGTH

AUC (bars start at 0.75)

256 to 512 points



0.824

513 to 765 points



0.876

766 to 1024 points



0.908

WHERE THE POINT SITS

AUC (bars start at 0.75)

30–43% of the way in



0.881

43–57%



0.869

57–70%



0.866

HOW VISIBLE THE BREAK IS

Breaks ranked in the top half

Quiet: under 0.15 SD



57%

Visible: over 0.15 SD



88%

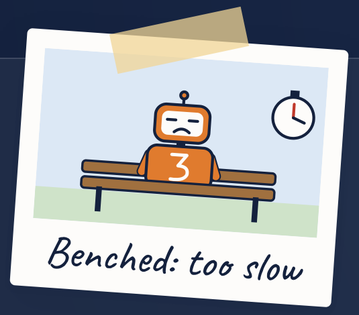
Missed breaks are quiet: their average or spread moves a median 0.12 SD (caught ones: 0.40). **False alarms are restless no-breaks**: 0.22 SD (typical: 0.11).

DECISION

The trade-off I chose

TRY3 · BEST SCORE, BENCHED

0.881



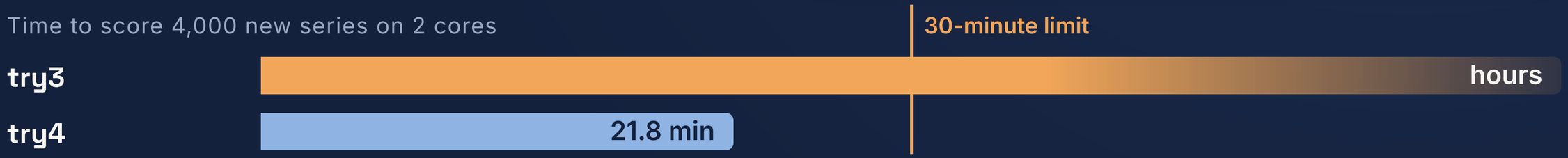
51 models plus corrections. About a day to train, hours to score new series: breaks the 30-minute rule.

TRY4 · NERFED, AND FINAL

0.876

3 tree models, 136 measurements. About 22 minutes on 2 cores with 2.6 GB of memory. Keeps **99.4%** of try3's score.

Time to score 4,000 new series on 2 cores



LIMITATIONS

What try4 can’t do

Quiet breaks slip through

43% of breaks that move the average or spread by under 0.15 SD land in the bottom half.

Short series are harder

0.82 on the shortest third (256–512 points) against 0.91 on the longest.

Nerfed by the rules

The 30-minute limit costs about 0.005: 0.876 for try4 against 0.881 for try3.

Picks aren’t “the truth”

Only 39 of the 100 columns are picked in all 5 rounds. Many stand in for each other, so read rankings loosely.

A ranking, not a probability

The 3 models’ ranks are averaged, so the output orders series but isn’t a calibrated chance, and all test series are scored in one run.

Built for this data

Designed around these series, untested on other data. A 4,000-series score also wobbles by about ±0.005.

REPRODUCIBILITY

Rerun it from the raw files with 3 commands

```
# 1 raw files → the 35 measurement tables and the noise priors
python try4/build_features.py --data data --work try4_run --workers 4

# 2 train and save the 15 models (3 models × 5 rounds)
python try4/train.py --work try4_run --out try4/model

# 3 score new series → submission.csv
python try4/predict.py --series data/test_series.parquet --ref try4_run --out predict_try4 --csv submission.csv --workers 2
```

CODE AND MODELS

github.com/blitzict/ku-sbb-2026, folder try4: code, the 15 saved models, and inference.ipynb.

SAME NUMBERS AGAIN

Rescoring with the saved models gives **0.87234** held-out (0.87235 recorded at training).

ON KAGGLE'S LIMITS

inference.ipynb scored all 4,000 test series on 2 cores in 21.8 min with 2.6 GB.

ENVIRONMENT, SOURCES AND AI USE

What it ran on, and who helped

01

Environment

Python 3.12 · LightGBM 4.7.0 · XGBoost 3.4.1 · scikit-learn 1.9.1 · numpy 2.5.4 · pandas 3.1.0 · scipy 1.18.1, pinned in requirements-cpu-full.txt. Trained on my own PC; scored on Kaggle with 2 CPU cores and 8 GB.

02

Sources

Only the competition’s training files. No outside data, no pretrained models, no test labels. try3’s correction network also used made-up series built from training no-break series.

03

AI use

Claude Opus 5.5 (Anthropic) turned my ideas into code, sped up training on my 2 GPUs, ported the features to a CPU-only run, and helped build these slides and the scripts behind their numbers.

WHAT I BUILT

My contribution, and what I learned



01

Describe how it was made

Mechanism measurements (noise laws, restarts, rhythm, momentum) instead of just how it looks: 0.82 for the best one alone, against 0.62 for eyeballing.

02

One specialist

Noise priors plus zoomed-in crops: one model at 0.8748, 99.5% of the 51-model ensemble.

03

Nerf it on purpose

51 models → 3, about 5,100 measurements → 136, hours → 21.8 minutes, for −0.005.

Lesson: one good idea beat a week of tuning. Understanding *how* a series is made beat staring at it.

Questions?